| EXP.NO: 7 | Producer Consumer |
| --- | --- |
| DATE: 06/03/2026 | |

## AIM:

To implement the Producer-Consumer problem using semaphores, ensuring mutual exclusion and preventing buffer overflow (Full) or underflow (Empty) during process execution.

## ALGORITHM:

- **Initialize Controls**: Start with a lock (**m=1**), 5 empty slots (**e=5**), and 0 full slots (**f=0**).
- **Producer Step**:

  Check if space is available.

  Lock the system, fill one empty slot, increase the full count, and then unlock.
- **Consumer Step**:

  Check if items exist.

  Lock the system, remove one full item, increase the empty slot count, and then unlock.
- **Safety Check**:

  If **e=0**, production is blocked (Full).

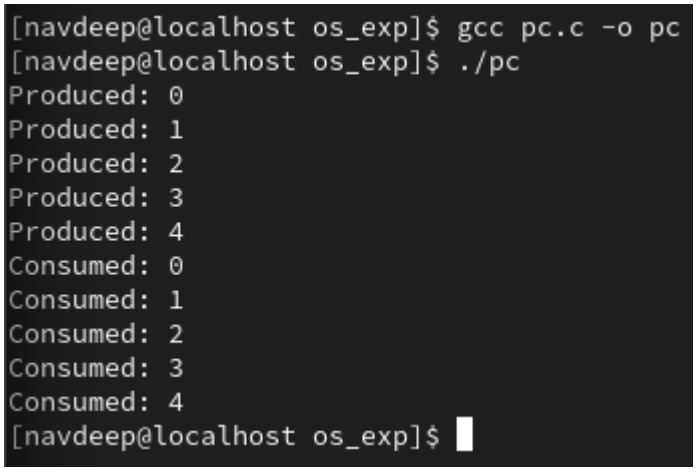
  If **f=0**, consumption is blocked (Empty).
- **Repeat**: Keep the process running in a loop until the user exits.

**CODE:**

```
#include <stdio.h>
#include <stdlib.h>

int m=1,f=0,e=5,x=0;

int wait(int s){ return --s; }
int signal(int s){ return ++s; }

void p(){
   m=wait(m); e=wait(e); f=signal(f);
   printf("\nProduced %d", ++x);
```

```
    m=signal(m);
}

void c(){
    m=wait(m); f=wait(f); e=signal(e);
    printf("\nConsumed %d", x--);
    m=signal(m);
}

int main(){
    int ch;
    while(1){
        printf("\n1.P 2.C 3.E: ");
        scanf("%d",&ch);
        if(ch==1){
            if(m==1 && e!=0) p();
            else printf("\nFull");
        }
        else if(ch==2){
            if(m==1 && f!=0) c();
            else printf("\nEmpty");
        }
        else exit(0);
    }
}
```

**OUTPUT:**




**RESULT:**

The program successfully simulates process synchronization. The producer is restricted from adding items when the buffer is full (e=0), and the consumer is restricted from removing items when the buffer is empty (f=0), maintaining data integrity within the shared resource.